

# Digital Electronics Solutions

## I. Number Systems

1. Convert the binary number 1101 to decimal: 13
2. Convert the octal number 127 to binary: 001 010 111 → 1010111
3. Convert the hexadecimal number 2F to decimal: 47
4. Binary equivalent of decimal 25: 11001
5. Convert 10011011 (binary) to hexadecimal: 9B
6. Convert decimal 345 to octal: 531
7. Convert hexadecimal A3 to binary: 10100011
8. Convert 111101 (binary) to octal: 75
9. Convert decimal 255 to hexadecimal: FF
10. Convert 0x1C to decimal: 28
11. Octal equivalent of binary 101101: 55

## II. Binary Codes

- 8421 BCD code for decimal 9: 1001
- Decimal 27 in BCD: 0010 0111
- BCD (Binary Coded Decimal) is a binary representation of decimal numbers where each digit is encoded separately using 4 bits.
- BCD is used instead of binary when accurate decimal representation is required in digital systems, e.g., digital clocks, calculators.
- Difference between BCD and binary:
- Binary represents entire numbers as a single binary value.
- BCD represents each decimal digit individually.
- Decimal 45 in BCD: 0100 0101
- Bits required for a single decimal digit in BCD: 4 bits
- Invalid digits in BCD: 1010 to 1111
- BCD is inefficient for arithmetic as it requires additional conversion steps.
- Advantages of binary codes: simple design, easy arithmetic, widely used in digital electronics.

## III. Boolean Algebra & Theorems

- De Morgan's Theorems:
- $(AB)' = A' + B'$
- $(A + B)' = A'B'$
- Dual of  $AB + AC$ :  $A + BC$

## IV. SOP, POS, Minterms, Maxterms

- Difference between SOP and POS:
- SOP: OR of AND terms, POS: AND of OR terms
- $F(A,B,C) = \Sigma(1,3,5,7)$  in SOP:  $A'B'C + A'B'C + A'BC + \dots$

- Maxterm expression for  $F(A,B,C) = \Pi(0,2,4,6)$ :  $(A+B+C)(A+B'+C)...$
- Minterm: product term representing a single '1' in truth table.
- Maxterm: sum term representing a single '0' in truth table.
- Convert  $Y = (A + B)(A' + C)$  to SOP:  $A'A C + AB C + ...$
- SOP form of  $F = A'BC + ABC$ :  $A'BC + ABC$
- $Y=(A+B)(A+C)(B+C')$  in standard POS:  $(A+B)(A+C)(B+C')$

## V. K-Maps & Minimization

- 3-variable K-map: label cells 0–7, variables A,B,C
- 4-variable K-map: label cells 0–15, variables A,B,C,D
- Simplify  $F(A,B,C)=\Sigma m(0,1,2,5,6,7)$  using K-map:  $F = A + B'C$
- Steps to minimize function using K-map:
  - Fill 1's
  - Group adjacent 1's in powers of 2
  - Derive simplified expression
  - Other K-map simplifications: see individual solutions
- Advantages: faster and visual simplification compared to Boolean laws
- Grouping in 4-variable K-map: adjacent 1's in 1,2,4,8 size groups
- 'Don't care' condition: input combination that can be used to simplify logic, e.g., output is irrelevant for that input.
- Real-life application of logic minimization: reducing cost of digital circuits.

## VI. Gray, Excess-3, Code Conversions

- Gray code of 1010: 1111
- Decimal 7 in Excess-3: 1010
- Purpose of Gray code: minimize errors in digital transitions
- Difference from binary: only one bit changes between successive numbers
- Excess-3 code 1000 to decimal: 5
- Gray code 1101 to binary: 1011
- Main application of Gray code: rotary encoders, position sensors
- Rule for binary  $\rightarrow$  Excess-3: add 3 to decimal, convert to 4-bit binary
- Reflected binary code: Gray code
- 3-bit Gray code truth table:
  - 000  $\rightarrow$  0
  - 001  $\rightarrow$  1
  - 011  $\rightarrow$  2
  - 010  $\rightarrow$  3
  - 110  $\rightarrow$  4
  - 111  $\rightarrow$  5
  - 101  $\rightarrow$  6
  - 100  $\rightarrow$  7
- Binary equivalent of Excess-3 1011:  $1000 \rightarrow 8-3 = 5$
- Convert BCD  $\rightarrow$  Excess-3: add 0011 to each BCD digit

## VII. Binary Addition, Subtraction, 1's & 2's Complement

- Binary addition rules:
  - $0+0=0$ ,  $0+1=1$ ,  $1+0=1$ ,  $1+1=10$
- Examples:
  - $1011+1101=11000$
  - $1111+0001=10000$
  - $100+110=1010$
  - $1010+0110=10000$  (decimal 16)
- Binary subtraction rules:
  - $0-0=0$ ,  $1-0=1$ ,  $1-1=0$ ,  $0-1=\text{borrow } 1$
- Examples:
  - $1101-1010=0011$
  - $1000-0011=0101$
  - $1001-0110=0011$
- 1's complement: invert all bits
- 1's complement subtraction: add minuend and 1's complement of subtrahend, add end-around carry if any
- 2's complement: invert all bits + 1
- 2's complement subtraction: add minuend and 2's complement of subtrahend, ignore carry if any